COMPUTER ARCHITECTURE & ORGANIZATION LECTURE (WEEK 2)

Interconnection Structures:

- Bus structures: Single bus, Multiple bus
- Instruction set Architecture: CISC, RISC
- Instruction and instruction sequencing:
- Register transfer notation, Instruction types.

Interconnection structures

- Computers has three main or basic modules: *Processor, Memory and I/O*
  - The three basic modules *must communicate with each other, therefore there must be paths for connecting these modules for them to communicate with each other.*

The collection of paths connecting the various modules is called the interconnection structure. The design of this structure will depend on the exchanges that must be made among modules.

■ Memory:

- Typically, a memory module will consist of N words of equal length.
- Each word is assigned a unique numerical address (0, 1, N-1).
- A word of data can be read from or written into the memory.
- The nature of the operation is indicated by read and write control signals.
- The location for the operation is specified by an address.

■ I/O module:

- From an internal (to the computer system) point of view, I/O is functionally similar to memory.
- There are two operations; **read and write.** Further, an I/O module may control more than one external device.

- We can refer to each of the interfaces to an external device as a ***port*** and give each a unique address (e.g., 0, 1, M-1). In addition, there are external data paths for input and output of data with an external device.
- Finally, an I/O module may be able to send ***interrupt signals*** to the processor.

■ **Processor:**

- The processor reads in instructions and data,
- Writes out data after processing, and uses control signals to control the overall operation of the system.
- It also receives *interrupt signals.*

The interconnection structure must support the following types of transfers:

■ **Memory to processor:** The processor reads an instruction or a unit of data from memory.

■ **Processor to memory:** The processor writes a unit of data to memory.

■ I**/O to processor:** The processor reads data from an I/O device via an I/O module.

■ **Processor to I/O:** The processor sends data to the I/O device.

■ I**/O to or from memory:** For these two cases, an I/O module is allowed to exchange data directly with memory, without going through the processor, using direct memory access(DMA).

Figure 1 below shows the Types of exchanges that are needed by indicating the major forms of input and output for each module type.

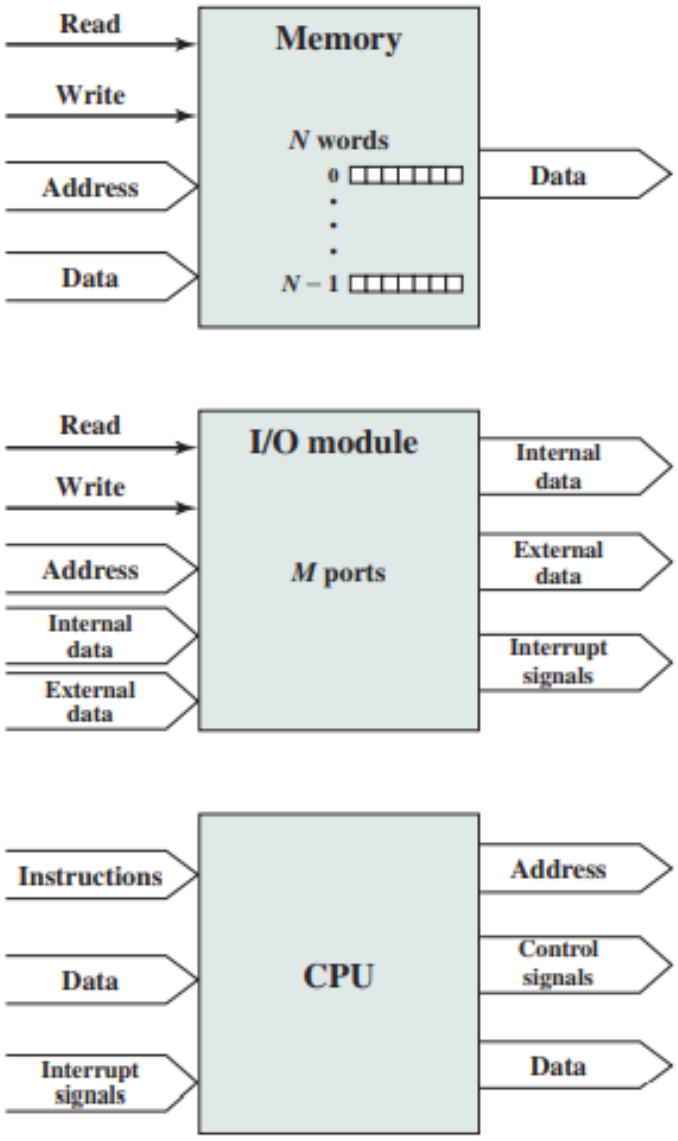

Figure 1: Types of exchanges that are needed by indicating the major forms of input and output for each module type.

Instruction set Architecture: CISC, RISC

The "best" way to design a CPU has been a subject of debate:

should the low-level commands be longer and powerful, using less individual instructions to perform a complex task ***(CISC),*** *or should the commands be shorter and simpler, requiring more individual instructions to perform a complex task* ***(RISC),*** *but allowing less cycles per instruction and more efficient code?*

- *CISC* (Complex Instruction Set Computer) and *RISC* (Reduced Instruction Set Computer) are two forms of CPU design.

- CISC uses a large set of complex machine language instructions, while RISC uses a reduced set of simpler instructions.

RISC and CISC is another axis along which we can organize computer architectures ad their instructions and how they are executed.

Reduced Instruction Set Architecture (RISC):

Making hardware simpler and to accommodate very basic instruction set that are made up of very few basic steps for loading, evaluation and storing.

The main idea behind this is to make hardware simpler by using an instruction set composed of a few basic steps for loading, evaluating, and storing operations just like a load command will load data, a store command will store the data.

Complex Instruction Set Architecture (CISC):

Making hardware complex in order accommodate very complex low level language statements that a single instruction will do all loading, operation and storing.

The main idea is that a single instruction will do all loading, evaluating, and storing operations just like a multiplication command will do stuff like loading data, evaluating, and storing it, hence it's complex.

Both approaches try to increase the CPU performance

Which of the design method of CISC and RISC is better in designing computer?

There is no "correct" answer: both approaches have pros and cons. ×86 CPUs (among many others) are CISC; ARM (used in many cell phones and PDAs), PowerPC, Sparc, and others are RISC.

- **RISC:** Reduce the cycles per instruction at the cost of the number of instructions per program.

- **CISC:** The CISC approach attempts to minimize the number of instructions per
- program but at the cost of an increase in the number of cycles per instruction.

Earlier when programming was done using assembly language, a need was felt to make instruction do more tasks because programming in assembly was tedious and error-prone ***due to which CISC architecture evolved*** but *with the uprise of high-level language dependency on assembly reduced RISC architecture prevailed.*

Characteristic of RISC –

1. Simpler instruction, hence simple instruction decoding.
2. Instruction comes undersize of one word.
3. Instruction takes a single clock cycle to get executed.
4. More general-purpose registers.
5. Simple Addressing Modes.
6. Fewer Data types.
7. A pipeline can be achieved.

Characteristic of CISC:

1. Complex instruction, hence complex instruction decoding.
2. Instructions are larger than one-word size.
3. Instruction may take more than a single clock cycle to get executed.
4. Less number of general-purpose registers as operations get performed in memory itself.
5. Complex Addressing Modes.

6. More Data types.

Example – Suppose we have to add two 8-bit numbers:

- **CISC approach:** There will be a single command or instruction for this like ADD which will perform the task.

- **RISC approach:** Here programmer will write the first load command to *load data in registers* then it will use a ***suitable operator*** and then it will ***store*** the result in the desired location.

So, add operation is divided into parts i.e. load, operate, store due to which RISC programs are longer and require more memory to get stored but require *fewer transistors due to less complex command.*

Difference:

	RISC	CISC
1.	Focus on software	Focus on hardware
2.	Uses only Hardwired control unit	Uses *both hardwired* and *microprogrammed control unit*
3.	Transistors are used for more registers	Transistors are used for storing complex Instructions
4.	Fixed sized instructions	Variable sized instructions
5.	Can perform only Register to Register Arithmetic operations	Can perform REG to REG or REG to MEM or MEM to MEM
6.	Requires more number of registers	Requires less number of registers
7.	Code size is large	Code size is small
8.	***An instruction executed in a single clock cycle***	***Instruction takes more than one clock cycle***
9.	An instruction fit in one word	Instructions are larger than the size of one word

Instruction and Instruction Types

Beyond the basic RISC/CISC characterization, we can classify computers by several characteristics of their instruction sets.

The instruction set of the computer defines the interface between software modules and the underlying hardware; ***the instructions define what the hardware will do under certain circumstances.***

Instructions can have a variety of characteristics, including:

- fixed versus variable length;
- addressing modes;
- numbers of operands;
- types of operations supported.

Word length

We often characterize architectures by their word length: 4-bit, 8-bit, 16-bit, 32-bit, and so on. In some cases, the length of a data word, an instruction, and an address are the same.

Particularly for computers designed to operate on smaller words, instructions and addresses may be longer than the basic data word.

Students should check out Little-endian vs. big-endian

We can also characterize processors by their instruction execution, a separate concern from the instruction set. A single-issue processor executes one instruction at a time. Although it may have several instructions at different stages of execution, only one can be at any particular stage of execution. Several other types of processors allow multiple-issue instruction. A superscalar processor uses specialized logic to identify at run time instructions that can be executed simultaneously.

The set of registers available for use by programs is called the programming model, also known as the programmer model. (The CPU has many other registers that are used for internal operations and are unavailable to programmers.)

INSTRUCTION SEQUENCING

Four types of operations

1. Data transfer between memory and processor registers.
2. Arithmetic & logic operations on data
3. Program sequencing & control
4. I/O transfers.

1)Register transfer notations (RTN)

R3<-[R1]+[R2]

- Right hand side of RTN-denotes a value.
- Left hand side of RTN-name of a location.

2)Assembly language notations (ALN)

Add R1, R2, R3

- Adding contents of R1, R2 & place sum in R3.

3)Basic instruction types-(4 types)

- **Three address instructions-**
- **Add A,B,C**

A, B-source operands

C-destination operands

- **Two address instructions-**
- **Add A,B**

Equivalent to : **B <-[A] + [B]**

- **One address instruction -**
- **Add A**

Add contents of A to accumulator & store sum back to accumulator.

- **Zero address instructions**

Instruction store operands in a structure called push down stack.

Instruction execution & straight line sequencing

- The processor control circuits use information in PC to fetch & execute instructions one at a time in order of increasing address.
  This is called straight line sequencing.

- Executing an instruction-2 phase procedures.

- 1st phase-**"instruction fetch"-**instruction is fetched from memory location whose address is in PC.
  This instruction is placed in instruction register in processor

- 2nd phase-**"instruction execute"-**instruction in IR is examined to determine which operation to be performed.

To be continued

References

William Stallings, Computer Organization and Architecture, 7th edition, Prentice Hall International, Inc., 2010.
https://www.geeksforgeeks.org/computer-organization-risc-and-cisc
https://www.sciencedirect.com/topics/computer-science
https://gowsalyaa24.wordpress.com/2011/04/28/instructions-and-instruction-sequencing/